

ENTREGA DE TAREAS

Sprint 3: Data Dictionary y mejora visual — FutbolWebApp

Pablo Arrazola Trebol

Prácticas — Data Control

13/11/2026

Resumen

Se ha continuado la documentación sobre los Data Dictionaries (DD) y se han llevado a FutbolWebApp las reglas de negocio de cada tabla: campos obligatorios, validaciones, valores por defecto, un contador automático de equipos por liga con su límite y el control del borrado en cascada. Además se ha mejorado el aspecto de la aplicación: tarjetas del Dashboard con color e icono, estilos en las listas, marca de campos obligatorios e iconos en los botones. El código está en el repositorio de GitHub, que es el enlace de entrega.

Repositorio (entrega): https://github.com/CidP34/proyectoGit — carpeta FutbolWebApp, rama master, etiqueta v3.0-sprint3

1. Qué es un Data Dictionary

Un Data Dictionary es una clase asociada a una tabla que reúne todas sus reglas de negocio: qué campos son obligatorios, qué valores son válidos, qué valores toma un registro nuevo, cómo se relaciona con otras tablas y qué pasa al guardar o borrar. Se guarda en un fichero .dd de la carpeta DDSrc del workspace.

En cada vista se crea un objeto de esa clase (DDO, Data Dictionary Object). Las vistas no contienen reglas: los formularios y las listas se enlazan a los DDO con Entry_Item y son los DDO los que buscan, validan, guardan y borran. Por eso una regla escrita una vez en el DD se cumple en todas las vistas, ya sea en la aplicación web o en una de Windows.

Las reglas se definen en el **DD Modeler** del Studio (abriendo el .dd) o escribiendo directamente el código; el Modeler genera el mismo código que se muestra en este documento.

2. Qué se define en un DD

Elemento	Para qué sirve y cómo se escribe
Relaciones	Tablas padre (Add_Server_File) e hijas (Add_Client_File). El Studio las genera a partir de las relaciones de las tablas.
Opciones de campo	Set Field_Option Field Tabla.Campo DD_Required to True. Otras: DD_DisplayOnly, DD_NoPut, DD_NoEnter, DD_CapsLock, DD_AutoFind, DD_FindReq, DD_Commit.
Etiquetas y ayuda	Field_Label_Long / Field_Label_Short (rótulo del campo) y Status_Help (texto de ayuda).
Validación de campo	Set Field_Validate_msg Field Tabla.Campo to (RefFunc(MiFuncion)). La función devuelve 0 si es válido; si no, lanza UserError y devuelve un valor distinto de 0.
Valores por defecto	Procedure Field_Defaults con Set Field_Changed_Value para rellenar un registro nuevo.
Validación del registro	Function Validate_Save: comprueba el registro completo antes de grabarlo.
Mantener otras tablas	Procedure Update (suma con los valores nuevos) y Procedure Backout (deshace con los valores antiguos).
Borrado	Cascade_Delete_State: True borra los hijos con el padre; False impide borrar un padre con hijos.

3. Qué pasa al guardar y al borrar

El orden de los eventos explica dónde va cada regla:

1. **Validaciones de campo** (Field_Validate_msg y DD_Required). Si alguna falla, no se llega a guardar.
2. Se abre la transacción y se bloquean las tablas.
3. Si se edita un registro existente, **Backout** con los valores antiguos (por ejemplo, la liga anterior de un equipo).
4. **Creating**, solo si el registro es nuevo.
5. **Update** con los valores nuevos, en el DD y en sus padres.
6. **Validate_Save**: última comprobación con todo ya actualizado. Si da error, se deshace la transacción entera.
7. Se graban el registro y sus padres modificados.

Al borrar se ejecuta **Backout** del registro y se aplica Cascade_Delete_State a sus hijos. Gracias a la pareja Backout/Update, los contadores se mantienen correctos tanto al crear y borrar como al mover un registro de un padre a otro.

4. Reglas de negocio de FutbolWebApp

Tabla	Reglas añadidas
liga	Nombre y País obligatorios · Equipos de solo lectura · EquiposMax entre 2 y 24, nunca menor que los equipos inscritos, 20 por defecto · No se borra una liga con equipos
equipo	Nombre obligatorio · Suma 1 a Liga.Equipos al guardar y resta al borrar o cambiar de liga · No se guarda si la liga supera su máximo · No se borra un equipo con jugadores o entrenador
entrenador	Nombre y Apellido obligatorios · Equipo opcional (entrenador libre) y único por el índice de MySQL
jugador	Nombre y Apellido obligatorios · Ayuda en Nacionalidad · Al borrarlo se borran sus posiciones (jugadorposicion)
posicion	Nombre obligatorio · No se borra una posición asignada a un jugador

Liga: validación y valor por defecto

```
Set Field_Option Field Liga.Equipos DD_DisplayOnly to True
Set Field_Validate_msg Field Liga.EquiposMax to (RefFunc(ValidarEquiposMax))

Procedure Field_Defaults
   Forward Send Field_Defaults
   Set Field_Changed_Value Field Liga.EquiposMax to 20
End_Procedure

Function ValidarEquiposMax Integer iField Integer iValor Returns Integer
   If (iValor < 2 or iValor > 24) Begin
      Send UserError "El máximo de equipos debe estar entre 2 y 24"
      Function_Return 1
   End
   If (iValor < Liga.Equipos) Begin
      Send UserError "La liga ya tiene más equipos inscritos que ese máximo"
      Function_Return 1
   End
   Function_Return 0
```

```
End_Function
```

Equipo: contador de equipos por liga y límite

La tabla liga ya trae los campos Equipos y EquiposMax (en el script, 3 equipos en cada liga). Hasta ahora nadie los mantenía; ahora lo hace el DD de Equipo:

```
Procedure Update
   Forward Send Update
   Add 1 to Liga.Equipos
End_Procedure

Procedure Backout
   Forward Send Backout
   Subtract 1 from Liga.Equipos
End_Procedure

Function Validate_Save Returns Integer
   Integer iError
   Forward Get Validate_Save to iError
   If (iError <> 0) Function_Return iError
   If (Liga.Equipos > Liga.EquiposMax) Begin
      Send UserError ("La liga " + Trim(Liga.Nombre) + " ya tiene el máximo de equipos (" +
String(Liga.EquiposMax) + ")")
      Function_Return 1
   End
   Function_Return 0
End_Function
```

Validate_Save se ejecuta después de Update, así que Liga.Equipos ya cuenta el equipo que se está guardando. Si se supera el máximo, el error deshace también el incremento.

Borrado en cascada

```
// cLigaDataDictionary, cEquipoDataDictionary, cPosicionDataDictionary
Set Cascade_Delete_State to False   // no se borra un padre con hijos

// cJugadorDataDictionary
Set Cascade_Delete_State to True    // borra sus filas de jugadorposicion
```

El código completo de los seis DD está en FutbolWebApp/DDSrc del repositorio.

Pruebas realizadas

Prueba	Resultado esperado
Guardar un jugador sin apellido	Error de campo obligatorio; no se guarda
Crear una liga nueva	EquiposMax aparece con 20
Poner EquiposMax = 1 o EquiposMax = 30	Error «entre 2 y 24»
Crear un equipo en LaLiga	LaLiga pasa de 3 a 4 equipos
Mover un equipo de LaLiga a Premier	LaLiga baja 1 y Premier sube 1
EquiposMax = 3 en una liga con 3 equipos y crear otro	Error «ya tiene el máximo»; el contador no cambia
Borrar una liga con equipos	Error; no se borra
Borrar un jugador con posiciones asignadas	Se borran el jugador y sus filas de jugadorposicion

Captura: error al guardar un equipo en una liga completa:

Captura: contador de equipos actualizado en SelectLiga:

5. Mejora visual

Mejora	Cómo se ha hecho
Tarjetas del Dashboard	Un color por tabla (colores de Data Control), icono, esquinas redondeadas, sombra y efecto al pasar el ratón. Toda la tarjeta es pulsable, no solo el texto.
Listas	Columnas numéricas alineadas a la derecha (Equipos, Máx.), columnas secundarias en gris y cursiva, texto cuando la lista está vacía.
Formularios	Asterisco rojo en los campos obligatorios del DD.
Botones	Iconos del tema en «Ver equipos» (buscar) y «Asignar posición» (añadir).
Textos	Títulos de las vistas en castellano: plural en las Select, singular en los Zoom.
Tema	Tema elegido con psTheme en WebApp.src, a partir de los que hay en AppHtml\CssThemes.

Los estilos propios van en AppHtml\CssStyle\application.css y se asignan a los objetos con psCSSClass:

```
.Tile .WebCon_Sizer {
    border-radius: 12px;
    box-shadow: 0 2px 6px rgba(0, 0, 0, 0.15);
    transition: transform 0.15s ease;
}
.Tile .WebCon_Sizer:hover { transform: translateY(-3px); }

.TileLiga.Tile .WebCon_Sizer    { background: #0B2A3B; }
.TileEquipo.Tile .WebCon_Sizer  { background: #00A676; }
.TileJugador.Tile .WebCon_Sizer { background: #00D18F; }

.ColNumero     { text-align: right; }
.ColSecundaria { color: #5A6B72; font-style: italic; }
```

```
Object oTileLiga is a cWebHtmlBox
    Set psCSSClass to "Tile TileLiga"
    Set pbServerOnClick to True
```

```
    Set psHtml to ('<div class="TileContenido" data-ServerOnClick="tile" data-OnClickParam="liga">' + ;
                   '<div class="Tile_Emoji">🏆</div>' + ;
                   '<div class="Tile_BigLabel">Ligas</div>' + ;
                   '<div class="Tile_SmallLabel">Competiciones</div></div>')
    ...
End_Object
```

Captura: Dashboard antes de la mejora:

Captura: Dashboard después de la mejora:

Captura: SelectLiga con las columnas numéricas y ZoomJugador con los obligatorios marcados:

6. Git y entrega

Rama	Contenido
feature/data-dictionary	Reglas en los seis .dd de FutbolWebApp/DDSrc
feature/mejoras-visuales	application.css, Dashboard.wo, vistas Select/Zoom y tema
develop	Integración de ambas ramas con merge --no-ff
master	Versión entregada; release del sprint con la etiqueta v3.0-sprint3

Se ha añadido un README.md en la raíz del repositorio con la descripción del proyecto, el modelo de datos, las reglas de negocio, la puesta en marcha y lo hecho en cada sprint, para que el enlace de GitHub sirva como documentación de entrega.

```
git checkout master
git merge --no-ff develop -m "Release Sprint 3: Data Dictionary y mejora visual"
git tag -a v3.0-sprint3 -m "Sprint 3 - Data Dictionary y mejora visual"
git push origin master --tags
git push origin --all
```

Captura: repositorio en GitHub con el README y la etiqueta v3.0-sprint3: